UNIVERSITÀ DEGLI STUDI DI SALERNO

Fondamenti di Visione Artificiale e Biometria

BasketAR

Gruppo 1

Emanuele Falanga Federica Graziuso Geraldine Montella

Dipartimento di Informatica

REPOSITORY github.com/GuardianMont/BasketAR_G1

ABSTRACT

Il Video Action Recognition applicato all'analisi della pallacanestro presenta complesse sfide metodologiche legate alla discriminazione di azioni a grana fine, alla marcata asimmetria distributiva delle classi e alle criticità nel tracciare elementi dinamici ad alta velocità e ridotta scala spaziale, come il pallone. I tradizionali modelli monolitici a classificazione diretta faticano a sintetizzare simultaneamente la postura dell'atleta e la micro-cinematica della palla, risultando spesso inefficaci nei task a più alto livello di dettaglio. Per superare tali limitazioni, questo lavoro presenta un framework modulare basato sul paradigma BNC (Backbone-Neck-Classifier) e strutturato secondo un'architettura a cascata gerarchica articolata su tre livelli predittivi. Per arricchire la rappresentazione spaziotemporale fornita dalle backbone, il sistema integra un modulo di tracking e detection dedicato basato su YOLO11m. La validazione sperimentale è condotta su un dataset di 5.721 clip video estrapolate da 6 partite distinte. La pipeline applica strategie di adattamento spaziale isotropo e campionamento temporale.

Keywords: Video Action Recognition; basket; classificazione gerarchica; fusione multimodale; Object Tracking; YOLO11m/26; Spatial-Temporal Backbones.

Contents

1 - Introduzione 4
1.1 - Definizione del task e del problema affrontato 4
2 - Stato dell'arte 5
2.1 - Action Recognition: panoramica del problema e dei principali approcci 5
2.2 - Action Recognition nel Basketball 6
2.3 - Architetture per il riconoscimento delle azioni: analisi delle componenti 7
2.4 - Tecniche per la gestione dello sbilanciamento dati 7
3 - Dataset 8
3.1 - Processo di labeling e modalità di annotazione dei dati 8
3.2 - Analisi esplorativa e statistica del dataset 8
3.2.1 - Partizionamento dei dati 8
3.2.2 - Distribuzione delle classi 8
3.3 - Caratteristiche temporali 9
3.4 - Classificazione gerarchica delle azioni 10
3.5 - Analisi delle rappresentazioni e delle relazioni tra classi mediante UMAP 10
3.5.1 - Sovrapposizione delle Partizioni e Prestazioni del Modello 11
4 - Preprocessing dei dati 12
4.1 - Campionamento temporale 12
4.2 - Preprocessing Spaziale 12
4.3 - Data augmentation 12
5 - Metodo e metodologia sperimentale 13
5.1 - Architettura e strategia di classificazione 13
5.1.1 - Modulo Backbone: estrazione delle feature 13
5.1.2 - Modulo Neck: modellazione temporale 13
5.1.3 - Modulo Head: aggregazione e classificazione 14
5.2 - Ricerca e selezione delle architetture candidate 14
5.2.1 - Feature probing: valutazione delle rappresentazioni 14
5.2.2 - Ricerca degli iperparametri dopo il probing 15
5.2.3 - Esperimento con tracker 16
5.2.4 - Metriche e valutazione 16
6 - Tracker palla-ferro e fusione con Video Swin 17
6.1 - Ruolo del tracking nel sistema 17
6.2 - Detector: dati, annotazione e fine-tuning 17
6.2.1 - Selezione dei frame 17
6.2.2 - Annotazione 18
6.2.3 - Scelta del detector: YOLO11m e YOLO26m 18
6.2.4 - Addestramento finale 19
6.3 - Detection e selezione temporale 20
6.3.1 - Inferenza 20
6.3.2 - Selezione dei candidati 20
6.4 - Allineamento temporale 20
6.5 - Feature di tracking 21
6.5.1 - Detection 21
6.5.2 - Cinematica della palla 21
6.5.3 - Relazione palla-ferro 21
6.5.4 - Aggregati di finestra 21
6.5.5 - Normalizzazione e valori mancanti 22
6.6 - Configurazioni di fusione 22
6.7 - Limiti e riproducibilità 22
7 - Risultati 23
7.1 - Risultati del probing 23

7.1.1 - ConvNext2D 24
7.1.2 - UniFormer-XXS 24
7.1.3 - VideoSwin 25
7.1.4 - X3D-S 25
7.1.5 - X3D-L 26
7.1.6 - Confronto 26
7.2 - Risultati senza tracker 27
7.3 - Risultati con tracker 29
8 - Discussione 30
9 - Conclusioni 31
Bibliography 31

1 - Introduzione

1.1 - Definizione del task e del problema affrontato

L'analisi automatica di filmati sportivi mediante tecniche di Computer Vision costituisce una sfida complessa, particolarmente pronunciata nel basketball video analytics. Il task affrontato in questo lavoro riguarda il Video Action Recognition e la segmentazione/classificazione di azioni di gioco a partire da registrazioni video non strutturate. La dinamicità del gioco, la rapidità delle transizioni di palla, le frequenti occlusioni tra i giocatori e la variabilità della durata delle azioni rendono complessa la corretta individuazione e segmentazione temporale degli eventi.

In questo lavoro proponiamo un approccio basato sull'architettura di deep learning formata da Backbone, Neck e Classifier, sfruttando backbone pre-addestrate su dataset di riferimento come *Kinetiks-400* [1].

Il dataset di riferimento comprende complessivamente **9 classi** . Nei primi esperimenti, i modelli addestrati per classificare direttamente tutte le categorie hanno mostrato prestazioni basse. Questo risultato ci ha portato a valutare un approccio diverso, basato sulla classificazione gerarchica, già utilizzata in letteratura [2] per affrontare problemi di classificazione complessi.

L'idea alla base di questo approccio è suddividere il problema in più compiti, ciascuno con un obiettivo specifico. Nel basket, infatti, non tutte le classi richiedono lo stesso tipo di analisi: per distinguere un passaggio da un tiro può essere sufficiente osservare il movimento complessivo dei giocatori, mentre per stabilire se un tiro è stato segnato o sbagliato è necessario riconoscere dettagli più specifici, come l'interazione tra la palla e il canestro.

Questo approccio ha portato sostanziali migliorie nelle prestazioni del modello finale e ci ha permesso di inferire la difficoltà della recognition di alcuni task, quali l'inferenza dell'esito di un tiro, rispetto ad altri, quali distinguere un tiro da un passaggio.

Questo lavoro è organizzato come segue:

- ***Capitolo 2 – Stato dell'arte***: presenta una rassegna della letteratura sul Video Action Recognition, analizzando le architetture convoluzionali 3D, i Video Transformer, i moduli di modellazione temporale e le applicazioni specifiche nello sport analytics e nel basket.
- ***Capitolo 3 – Dataset***: illustra il processo di annotazione e le scelte progettuali, descrive l'analisi esplorativa e statistica del dataset, definisce la struttura della classificazione gerarchica e analizza la separabilità delle rappresentazioni tramite UMAP.
- ***Capitolo 4 – Preprocessing***: descrive il campionamento temporale, il preprocessing spaziale e le strategie di data augmentation.
- ***Capitolo 5 – Metodologia***: descrive la pipeline BNC (Backbone, Neck, Head), i task gerarchici, il probing delle rappresentazioni, la ricerca degli iperparametri e il protocollo di valutazione.
- ***Capitolo 6 – Tracking e fusione delle feature***: presenta il rilevamento di palla e ferro, la costruzione delle feature di tracking e il loro allineamento con Video Swin.
- ***Capitolo 7 – Risultati***: presenta e analizza i risultati dei modelli visual-only e degli esperimenti con tracking.

2 - Stato dell'arte

2.1 - Action Recognition: panoramica del problema e dei principali approcci

Il Video Action Recognition (VAR) [3] mira a classificare o segmentare temporalmente azioni umane all'interno di sequenze video non strutturate, integrando l'analisi delle caratteristiche spaziali nei singoli fotogrammi con la dinamica temporale del movimento.

In letteratura, la formulazione del task si articola in due paradigmi principali:

- **Untrimmed Video Action Localization**: si applica a flussi video continui e non strutturati (es. l'intera trasmissione di un evento sportivo) [4], richiedendo al sistema di individuare contestualmente i confini temporali di inizio e fine dell'azione (**detection**) e di associarvi la corretta etichetta di classe [5].
- **Trimmed Clip-based Action Recognition**: presuppone che il flusso video originale sia già stato pre-segmentato in brevi intervalli temporali definiti (**clip**), ciascuno contenente una singola azione dominante [1], [6]. In questo scenario, la dimensione temporale viene condizionata a un budget fisso di fotogrammi (T), e l'obiettivo del modello si riduce alla stima della distribuzione di probabilità sulle classi target per l'intera clip.

Il presente lavoro si colloca rigorosamente nel secondo paradigma (**clip-level classification**), concentrandosi sulla caratterizzazione ad alta precisione di clip pre-delimitate relative alle fasi salienti del gioco.

Dalle hand-crafted feature ai modelli 2D: Storicamente, i primi tentativi di caratterizzazione del movimento si affidavano a descrittori calcolati manualmente (*hand-crafted features*), tra cui i *Dense Trajectories* [7] e l'analisi del flusso ottico (*Optical Flow*) [8]. Sebbene efficaci in scenari controllati, questi approcci soffrivano di una spiccata sensibilità a variazioni ambientali, quali cambiamenti di illuminazione, occlusioni e movimenti incontrollati della telecamera. L'introduzione delle reti neurali convoluzionali (CNN) 2D come *ResNet* [9] e *ConvNeXt* [10], hanno segnato il passaggio all'apprendimento profondo. In questo paradigma iniziale, i fotogrammi venivano elaborati in maniera indipendente e le rappresentazioni estratte venivano successivamente aggregate tramite *temporal pooling* (media o massimo). Tuttavia, tale approccio non consentiva di modellare l'ordine sequenziale e la reale continuità temporale del movimento.

Per superare l'incapacità delle architetture 2D di cogliere la dimensione temporale, la ricerca ha introdotto le *3D CNN*, estendendo i filtri convoluzionali direttamente lungo il volume spaziotemporale formato da fotogrammi consecutivi. La pietra miliare di questa transizione è rappresentata da *I3D* (*Inflated 3D ConvNet* [11]) , che ha dimostrato la possibilità di "gonfiare" (*inflate*) i filtri 2D pre-addestrati su ImageNet trasformandoli in filtri 3D capaci di apprendere rappresentazioni spazio-temporali congiunte. Nonostante l'elevata capacità espressiva, le 3D CNN tradizionali presentavano costi computazionali elevati. Per ovviare a questo limite, *X3D* [12] ha ridefinito l'efficienza delle reti 3D partendo da un'architettura base estremamente compattata ed espandendola in modo bilanciato lungo più dimensioni (risoluzione spaziale, frequenza di campionamento temporale, canali e profondità), riducendo drasticamente il numero di FLOPs e l'uso di memoria a parità di accuratezza predittiva.

Il successivo cambio di paradigma è stato guidato dall'estensione dei meccanismi di *Self-Attention* al dominio video, superando i vincoli di campo recettivo locale tipici delle convoluzioni. Tra i modelli principali, *Video Swin Transformer* [13] adatta l'architettura Swin ai video estendendo le finestre di attenzione (*Shifted Windows*) allo spazio tridimensionale. Calcolando l'attenzione all'interno di volumi locali compatti, il modello garantisce un'ottima scalabilità al crescere della lunghezza della sequenza. Parallelamente, per coniugare l'efficienza locale della convoluzione con la capacità espressiva dell'attenzione globale, *UniFormer* [14] propone una struttura ibrida che impiega blocchi convoluzionali 3D negli strati superficiali (per abbattere la ridondanza spaziale locale) e blocchi di *Global Self-Attention* negli strati più profondi, cogliendo efficacemente le relazioni e le dipendenze a lungo termine lungo l'intero video.

2.2 - Action Recognition nel Basketball

L'analisi delle azioni nella pallacanestro presenta complessità superiori rispetto ai benchmark generici (es. Kinetics-400 [1]), a causa dell'elevata velocità dei movimenti, delle continue occlusioni in area e delle ridotte dimensioni del pallone nelle riprese televisive.

Per affrontare tali sfide, la ricerca si avvale di dataset dedicati (SpaceJam, Basketball-51) e di dati di tracciamento ad alta precisione (NBA e NCAA) per addestrare modelli di video analytics.

Object Tracking e fusione multimodale Poiché le sole feature estratte da una 3D-CNN non sempre catturano dettagli fine-grained determinanti (es. l'esito di un tiro), diversi approcci integrano modelli di object detection basati su YOLO [15] per tracciare la traiettoria del pallone nel tempo. La fusione tra feature visive generali e coordinate spaziali del pallone incrementa la capacità discriminativa della rete negli eventi critici.

2.3 - Architetture per il riconoscimento delle azioni: analisi delle componenti

I moderni sistemi di Video Action Recognition vengono frequentemente strutturati secondo un paradigma modulare suddiviso in tre componenti disaccoppiate: **Backbone, Neck e Head.** Questa separazione concettuale non solo facilita il riuso di moduli pre-addestrati, ma consente di ottimizzare in modo indipendente l'estrazione visiva, la modellazione temporale e la decisione finale.

La Backbone come estrattore di feature spazio-temporali Il primo stadio dell'elaborazione è affidato alla *Backbone*, il cui ruolo primario è agire da *feature extractor*. Ricevendo in ingresso il volume video grezzo, la *Backbone* riduce progressivamente le dimensioni spaziali (H, W) tramite operazioni di pooling o proiezioni di attenzione, convertendo i pixel in vettori di rappresentazione ad alta densità semantica. Parallelamente, preserva o sottocampiona in modo controllato la dimensione temporale T, generando una sequenza di feature che mantiene l'evoluzione cronologica dell'azione ma ad un livello di astrazione nettamente superiore.

Il Temporal Neck per la modellazione delle dipendenze sequenziali Poiché la sola estrazione di caratteristiche visive fotogramma per fotogramma risulta insufficiente a cogliere il contesto di un'azione complessa, si interpone un modulo temporale dedicato, comunemente definito *Neck*. Il suo compito fondamentale è mettere in relazione i vettori estratti dalla *Backbone* lungo l'asse temporale, superando i limiti dell'elaborazione isolata. La letteratura propone:

- **Bi-GRU (Bidirectional Gated Recurrent Units)** [16]: analizza la sequenza temporale considerando sia i fotogrammi precedenti che quelli successivi. Rispetto alle classiche LSTM, la struttura a due porte (*Reset* e *Update*) riduce il carico computazionale mantenendo un'ottima capacità di modellazione temporale.
- **TCN (Temporal Convolutional Networks)** [17]: sfruttano convoluzioni 1D causali e dilatazioni esponenziali. Offrono diversi vantaggi: calcolo parallelo in fase di addestramento, stabilità grazie alle connessioni residue (evitando il problema del *vanishing gradient*) e un ampio campo recettivo (*receptive field*).
- **Temporal Soft-Attention** [18]: assegna pesi dinamici a ciascun fotogramma, concentrando la capacità rappresentativa sui soli momenti salienti dell'azione (es. il rilascio del pallone o l'impatto con il canestro).

La Head per l'aggregazione e la classificazione finale A valle del modulo temporale si colloca la *Head*, che funge da aggregatore e classificatore finale. Il suo obiettivo è contrarre la sequenza di vettori temporali elaborata dal *Neck* in un unico vettore di rappresentazione compatto per l'intera clip. Tale contrazione avviene solitamente mediante operazioni di *mean pooling* (media temporale) o mediante pooling pesato dall'attenzione. Si occupa anche di proittera questo vettore finale nello spazio dei *logits* associati alle classsi target, fornendo le distribuzioni di probabiltà per la classificazione dell'azione.

2.4 - Tecniche per la gestione dello sbilanciamento dati

Nei video sportivi, la distribuzione delle classi risulta intrinsecamente sbilanciata: le fasi di non-gioco, le situazioni stazionarie (idle) e i passaggi ordinari costituiscono la maggior parte delle clip, mentre i tiri e i relativi esiti rappresentano una frazione minoritaria.

Per contrastare il bias predittivo verso le classi maggioritarie, la letteratura adotta funzioni di perdita modificate, tra cui la Focal Loss [19]:

$$\mathrm{FL}(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t)$$

Dove p_t è la probabilità stimata per la classe corretta, α_t bilancia il peso tra le classi e γ è il parametro di focalizzazione (*focusing parameter*).

Il termine $(1 - p_t)^{\gamma}$ riduce drasticamente l'impatto degli esempi facili e già ben classificati (come le fasi di inattività o i passaggi di routine), costringendo la rete a concentrarsi sui casi più rari e complessi (come i tiri contestati o i tiri liberi).

3 - Dataset

3.1 - Processo di labeling e modalità di annotazione dei dati

Il dataset è stato costruito attraverso un processo di annotazione manuale frame-accurate condotto su filmati ad alta risoluzione di partite di pallacanestro, con l'obiettivo di garantire la massima coerenza ed omogeneità visivo-semantica delle etichette.

Il protocollo impone la delimitazione esatta di ogni evento mediante l'individuazione puntuale del fotogramma iniziale (start frame) e finale (end frame). Per preservare la qualità del dataset ed evitare l'introduzione di rumore nelle etichette.

Le azioni che presentano ambiguità visiva, occlusioni prolungate, atleti fuori inquadratura o riprese parziali vengono escluse quando non risultano chiaramente riconoscibili con sufficiente certezza. Le occorrenze di azioni sovrapposte o temporalmente contigue (es. passaggio e tiro adiacenti) vengono iscritte in finestre temporali distinte tramite ispezione fotogramma per fotogramma.

3.2 - Analisi esplorativa e statistica del dataset

3.2.1 - Partizionamento dei dati

Il dataset si compone di **5.721 clip video** associate a **6 valori distinti di** `video_id` nel manifest. La tabella seguente riporta la distribuzione complessiva delle clip fra le partizioni; quella successiva dettaglia gli ID sorgente assegnati a ciascuno split.

Partizione	Numero di clip
Training	4.242
Validation	540
Test	939
Totale	5.721

Partizione	Video ID nel manifest	Numero di clip
Train	`basket_2nd`	1.130
Train	`prima_parte`	534
Train	`psa_corato`	1.176
Train	`video1`	1.402
Validation	`seconda_parte`	540
Test	`psa_converted`	939
Totale	6 video ID	5.721

La ripartizione è quindi definita a livello di `video_id`, ma non corrisponde a una separazione completa per partita: `prima_parte` (train) e `seconda_parte` (validation) sono due parti della stessa partita. La validation non misura pertanto la generalizzazione a una partita completamente assente dal train. Questa condivisione del contesto può influenzare il confronto fra le partizioni, ma non dimostra da sola la presenza di *data leakage*. Per valutare la generalizzazione a partite mai viste, le due parti dovrebbero essere assegnate allo stesso split e gli esperimenti ricalcolati; con la ripartizione attuale, i risultati vanno interpretati come ottenuti su video ID distinti, con possibile sovrapposizione del contesto della partita.

3.2.2 - Distribuzione delle classi

La distribuzione delle istanze evidenzia un marcato sbilanciamento. La classe passaggio rappresenta la categoria maggioritaria con il 43,8% delle clip (2.505), seguita dalle classi di supporto non-gioco (1.503) e idle (855). Le categorie legate al tiro sono fortemente sottorappresentate, raggiungendo i minimi con tiroLibero0

(82 clip) e tiroDaTre1 (58 clip). Tale asimmetria rende necessaria l'adozione di funzioni di costo bilanciate (es. Focal Loss) durante l'ottimizzazione dei modelli.

Classe	Train	Validation	Test
passaggio	1.787	212	506
non-gioco	1.177	170	156
idle	645	93	117
tiroDaDue0	197	21	48
tiroDaDue1	128	11	50
tiroDaTre0	111	12	29
tiroDaTre1	46	3	9
tiroLibero0	62	7	13
tiroLibero1	89	11	11
Totale	4.242	540	939

3.3 - Caratteristiche temporali

L'analisi delle durate evidenzia un'elevata eterogeneità temporale tra le diverse categorie di azione:

La durata media varia da un minimo di 0,50 s per la classe **passaggio** a un massimo di 2,24 s per la classe **non-gioco**. Le azioni di **tiro** presentano durate medie comprese tra *1,28 s e 1,88 s*. 1.799 clip (pari al 31,4% del totale) hanno una durata inferiore a 0,5 s.

2.078 clip (pari al 36,3% del totale) contengono meno di 16 fotogrammi a una frequenza di campionamento di 30 FPS.

Tale variabilità richiede l'adozione di strategie mirate di campionamento temporale e di padding dinamico per preservare la struttura informativa delle sequenze più brevi senza compromettere quelle più estese.

Classe	Media(s)	Mediana(s)	Min(s)	Max(s)
idle	0,72	0,52	0,15	2,50
non-gioco	2,24	2,21	0,22	5,00
passaggio	0,50	0,47	0,15	2,01
tiroDaDue0	1,28	1,27	0,30	2,64
tiroDaDue1	1,33	1,26	0,33	2,77
tiroDaTre0	1,88	1,84	0,91	3,19
tiroDaTre1	1,75	1,71	0,99	3,51
tiroLibero0	1,88	1,91	0,57	2,98
tiroLibero1	1,70	1,57	1,10	3,67

3.4 - Classificazione gerarchica delle azioni

La classificazione diretta a 9 classi (*flat classification*) ha mostrato limiti prestazionali dovuti sia allo sbilanciamento sia al diverso livello di dettaglio visivo richiesto dai vari task. Per ovviare a tali criticità, il problema è stato strutturato secondo un approccio di classificazione gerarchica articolato su tre sotto-task a cascata:

- ***Fase 1*** (p_1) **–** ***Riconoscimento della Macro-Azione***: il modello analizza la clip video e la classifica in tre macro-categorie: **Passaggio, Tiro e Altro** (quest'ultima comprendente le classi idle e non-gioco). L'obiettivo è separare la dinamica generale del gioco senza considerare i dettagli specifici delle tipologie di tiro.
- ***Fase 2*** (p_2) **–** ***Riconoscimento della tipologia di tiro***: il modello opera esclusivamente sul sottoinsieme delle clip identificate come Tiro, classificando l'azione in una delle tre categorie: **Tiro da due punti, Tiro da tre punti o Tiro libero**.
- ***Fase 3*** (p_3) **–** ***Riconoscimento dell'esito del tiro***: il terzo modello esegue una classificazione binaria (**Segnato vs Sbagliato**) sul medesimo sottoinsieme delle clip di tiro. Questo sotto-task richiede di focalizzare la capacità rappresentativa dell'architettura sui dettagli visivi dell'interazione palla-canestro.

3.5 - Analisi delle rappresentazioni e delle relazioni tra classi mediante UMAP

Per valutare la struttura dello spazio delle feature estratte dalla backbone e l'allineamento distributivo tra le partizioni di dati (Train, Validation e Test), è stata condotta un'analisi di UMAP (Uniform Manifold Approximation and Projection).

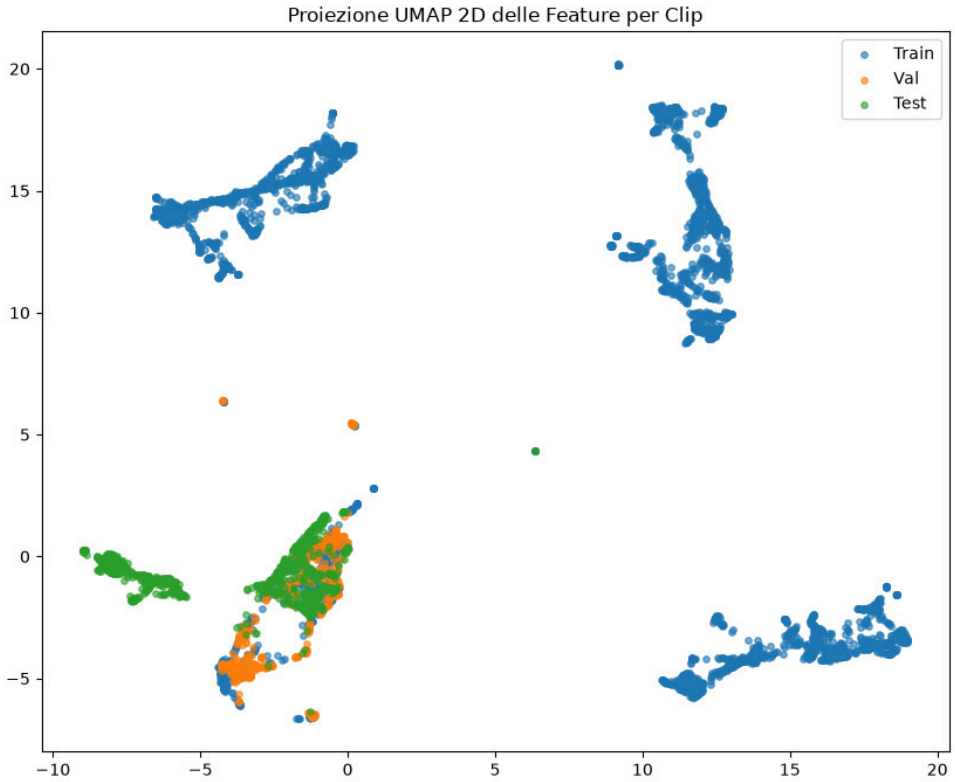


Figure 1: Proiezione UMAP 2D delle feature estratte per clip video, suddivise tra le partizioni di Train, Validation e Test.

È stato scelto UMAP in quanto, a differenza di PCA, studia le correlazioni su manifold, tentando di preservare la struttura topologica di uno spazio ad alta dimensionalità, evitando di proiettarla su sottospazi lineari, che potrebbe corrompere le relazioni di vicinanza tra punti degli embedding.

Come illustrato in Figure 1: i vettori di rappresentazione del ***Train set*** non formano un continuum omogeneo, ma si articolano in quattro macro-cluster ben distinti.

Poiché il partizionamento è stato eseguito a livello di video sorgente, questa separazione evidenzia un sensibile domain shift legato alla specifica partita (plausibilmente illuminazione, inquadratura, colori di campo e divise). Inoltre la sovrapposizione quasi perfetta della fold del ***Validation set*** con una regione del Train set e il parziale isolamento del ***Test set*** suggeriscono due cose:
1. Possibile data leakage nel validation set
2. Forte domain shift nel test set

Queste due ipotesi verranno validate e riprese per spiegare alcuni risultati anomali nella sezione predisposta. Infine, dall'analisi topologica è evidente che ci sia una forte dipendenza tra le clip video e le partite a cui queste appartengono, non garantendo l'ipotesi di IID (Indipendence and Identical Distribution), non permettendo una ristrutturazione delle fold con random shuffling o k-fold, le quali rischierebbero di introdurre del data leakage dato dalla correlazione tra clip e partita.

3.5.1 - Sovrapposizione delle Partizioni e Prestazioni del Modello

L'analisi dello spazio latente fornisce una spiegazione chiara delle prestazioni rilevate:

- ***Validation set***: si sovrappone in modo quasi totale al cluster di Train inferiore-centrale, indicando un'elevata affinità visiva con uno dei filmati di addestramento. Tale allineamento spiega le elevate prestazioni ottenute in fase di validazione.
- ***Test set***: presenta una struttura bimodale. Circa la metà delle istanze condivide il cluster inferiore-centrale con Train e Val, garantendo una buona generalizzazione su questo sottoinsieme. Tuttavia, una quota rilevante forma un sub-cluster isolato all'estrema sinistra, del tutto privo di riscontro nei dati di addestramento.

La presenza di tale regione non esplorata in fase di training è la causa primaria del calo prestazionale riscontrato sul Test set, spiegando in modo coerente il comportamento e i limiti del modello.

4 - Preprocessing dei dati

La pipeline di preprocessing trasforma i flussi video grezzi e le annotazioni temporali nei tensori di input richiesti dai modelli di Video Action Recognition 3D. L'obiettivo primario è uniformare le dimensioni spaziali e temporali delle clip, massimizzando la densità informativa e preservando la coerenza geometrica e cinematica dei gesti atletici

4.1 - Campionamento temporale

Per gestire le diverse durate delle sequenze sono previste tre strategie:

- ***Campionamento uniforme***: Seleziona un numero fisso di frame ($N = 16$) equispaziati. Se la clip ha meno di 16 frame, il riempimento avviene tramite replicazione dell'ultimo frame.
- ***Campionamento a frame rate fisso***: Seleziona i frame a 15 FPS target in base al frame rate originale, mantenendo inalterata la scansione temporale reale.
- ***Campionamento non lineare*** (***EdgeDense***): Distribuisce un budget fisso di N frame concentrando la densità di campionamento agli estremi temporali (fase preparatoria ed esecutiva/finale dell'azione) e diradandola al centro. Questa tecnica massimizza l'informazione utile senza creare discontinuità o perdere del tutto il contesto centrale. Il profilo è regolato dal parametro di distorsione cosinusoidale $\alpha \in [0, 1]$
  - $\alpha = 0$ equivale al campionamento uniforme,
  - $\alpha = 1$ massimizza la densità ai margini.

  Rispetto a un troncamento netto, edge_dense preserva una visione d'insieme dell'intera clip senza generare discontinuità o perdere del tutto il contesto centrale.

4.2 - Preprocessing Spaziale

Poiché i filmati sorgente del dataset presentano un formato widescreen rettangolare (tipicamente $16 : 9$), è necessario applicare una trasformazione geometrica per ricondurre ciascun fotogramma di dimensione originale $H \times W$ alla dimensione quadrata target $S \times S$. Nel modulo di preprocessing sono state implementate e confrontate due modalità alternative di adattamento spaziale:

- ***Stretch***:esegue un ridimensionamento diretto e non isotropo del fotogramma alla dimensione target $S \times S$ tramite interpolazione bilineare con filtro anti-aliasing. È efficiente dal punto di vista computazionale e sfrutta tutta l'area disponibile, ma può deformare le proporzioni degli oggetti.
- ***Letterbox*** :preserva rigorosamente l'aspect ratio originale della clip mediante un ridimensionamento uniforme (isotropo), colmando lo scarto residuo con bande nere simmetriche (zero-padding).Garantisce la conservazione delle proporzioni corporee dei giocatori, della forma del pallone e della traiettoria di tiro.

4.3 - Data augmentation

Per preservare la coerenza spazio-temporale delle azioni ed evitare distorsioni che possano degradare l'apprendimento, le trasformazioni spaziali vengono applicate in modo identico ed uniforme su tutti i fotogrammi dell'intera sequenza:

- ***Random Horizontal Flip***: inverte casualmente l'orientamento orizzontale dei frame.
- ***Color Jitter***: applica variazioni stocastiche alle componenti di luminosità, contrasto, saturazione e tonalità.

Altri metodi di augmentation sono stati valutati ma considerati non adatti alla modellazione proposta:

- ***Random Temporal Shift***: sebbene molto usato in task di computer vision simili, comprometterebbe alcuni campionamenti: *edge dense* per esempio ha come idea di base quella di campionare con alta probabilità i frame iniziali e finali della clip, spesso coincidenti con momenti cardine delle azioni di basket. Un temporal shift ne comprometterebbe l'efficacia.
- ***Random Vertical Flip***: non coerente con l'inquadratura della partita.
- ***Random Crop***: avrebbe rischiato di tagliare molte azioni agli estremi dell'inquadratura, in particolare i tiri.

5 - Metodo e metodologia sperimentale

5.1 - Architettura e strategia di classificazione

La metodologia sviluppata in questo lavoro nasce dalla necessità di affrontare due sfide principali evidenziate dall'analisi del dataset: la diversa natura dei sotto-task necessari per identificare e caratterizzare le azioni nel basket e la spiccata sensibilità dei modelli alle variazioni di dominio (**domain shift**).

Per rispondere a tali criticità, il sistema adotta un approccio basato su due pilastri interconnessi:

1. ***Pipelining modulare BNC* (*Backbone, Neck, Head*)**: la separazione concettuale tra estrazione delle feature spaziali/spazio-temporali (**Backbone**), modellazione del contesto temporale (**Neck**) e classificazione finale (**Head**) consente di isolare il contributo di ciascun componente e di sperimentare diverse combinazioni architetturali.
2. ***Architettura a cascata gerarchica***: il problema complessivo viene suddiviso in tre moduli predittivi gerarchici, nei quali ciascun classificatore è specializzato su un sotto-task specifico.

La pipeline computazionale integra inoltre le strategie di preprocessing (campionamento temporale uniforme, a FPS fisso ed **EdgeDense**) e la **Focal Loss** per mitigare l'impatto dello sbilanciamento delle classi.

5.1.1 - Modulo Backbone: estrazione delle feature

La **Backbone** agisce come estrattore di caratteristiche visive. Riceve in ingresso il tensore video V e produce una sequenza di vettori di feature $F \in \boldsymbol{R}^{B\times T'\times D_{\{\text{backbone}\}}}$, riducendo le dimensioni spaziali (H, W) tramite **Global Average Pooling** (GAP) e preservando o sottocampionando l'asse temporale T':

$$\boldsymbol{F}(b,t) = \frac{1}{H' \times W'} \sum_{h=1}^{H'} \sum_{w=1}^{W'} \boldsymbol{M}(b,:,t,h,w)$$

Architetture integrate:

- **X3D-S (Conv3D Lightweight)**:Efficiente a basso costo computazionale. Rimuovendo la testa di classificazione originale, permette di estrarre feature spazio-temporali mantenendo la risoluzione temporale inalterata $T_{\text{out}} = T_{\text{in}}$ e restituendo un tensore denso di dimensione $D_{\text{out}} = 192$.
- **X3D-L (Conv3D Deep)**: Estensione ad alta capacità dell'architettura X3D-S con proiezione a 432 canali. Viene impiegata quando è richiesta una maggiore capacità espressiva per catturare dettagli visivi e movimenti complessi, a fronte di un maggior carico computazionale.
- **ConvNeXt-Tiny (2D Frame-by-Frame Spatial Extractor)**: architettura convoluzionale 2D. OOpera una caratterizzazione puramente spaziale e statica fotogramma per fotogramma: il tensore di input viene risagomato combinando il batch e il tempo per poi essere processato in modo indipendente da blocchi 2D.Un'operazione di Global Average Pooling mappa ciascun frame in un vettore di dimensione $D_{\text{out}} = 768$.
- **Video Swin (3D Vision Transformer)**:Transformer spazio-temporale basato sul meccanismo di attenzione a finestre traslate (Shifted Windows). È impiegato per catturare contesti visivi complessi e relazioni a lungo raggio tramite Self-Attention. Il dimezzamento della risoluzione temporale ($T' = \frac{T}{2}$) durante il patch embedding consente di abbattere la complessità computazionale del Transformer senza perdere la dinamica globale.
- **UniFormer-XXS (Ibrido Conv3D / Transformer)**: architettura ibrida che combina blocchi convoluzionali locali (**C-Block**) negli strati superficiali per ridurre la ridondanza spaziale e blocchi di **Global Self-Attention** (**SA-Block**) guidati da **Dynamic Positional Encoding** (DPE) negli strati profondi.

5.1.2 - Modulo Neck: modellazione temporale

Il **Temporal Neck** riceve in ingresso il tensore di feature $\boldsymbol{F} \in \boldsymbol{R}^{\{B\times T'\times D_{\text{in}}\}}$ ed elabora le dipendenze temporali tra i fotogrammi, producendo un tensore contestualizzato $\boldsymbol{H} \in \boldsymbol{R}^{\{B\times T'\times D_{\text{out}}\}}$:

Varianti implementate:

- **Identity Neck**: modulo nullo che restituisce l'input immutato, funge da controllo per quantificare l'effettivo valore aggiunto apportato dai moduli temporali complessi rispetto alle sole feature della backbone.
- **BiGRU Neck (Recurrent Neural Network)**: rete ricorrente bidirezionale che modella la sequenza considerando sia il contesto passato che quello futuro. L'adozione del **packing** dinamico delle sequenze evita che i fotogrammi di **padding** alterino lo stato nascosto.
- **TCN Neck**: modulo a convoluzioni 1D dilatate ($d = 2^i$) e connessioni residue. Espande il campo recettivo temporale (*receptive field*) e consente il calcolo parallelo in fase di addestramento. Nell'implementazione adottata le convoluzioni non sono causali.

5.1.3 - Modulo Head: aggregazione e classificazione

Il modulo Head riduce la sequenza di feature temporali $\boldsymbol{H} \in \boldsymbol{R}^{B \times T \times D_{\text{neck}}}$ ad un unico vettore di rappresentazione per l'intera clip video, proiettandolo infine sui logits della classe di azione $\boldsymbol{Z} \in \boldsymbol{R}^{B \times K}$

Varianti implementate:

- **Mean Pooling Classifier Head**: calcola la media semplice dei vettori di feature lungo l'asse temporale T', presupponendo che ciascun istante contribuisca in egual misura all'identificazione dell'azione.
- **Attention Head** : assegna pesi dinamici $\alpha_{b,t}$ a ciascun fotogramma tramite un meccanismo di **Soft-Attention**, permettendo alla rete di concentrare la propria capacità rappresentativa sui soli momenti salienti della sequenza (es. rilascio della palla o impatto con il canestro) e azzerando i pesi dei fotogrammi di **padding** tramite mascheramento prima del calcolo della **Softmax**:

$$e_{b,t} = \boldsymbol{W}_2 \cdot \tanh\big(\boldsymbol{W}_1 \cdot \boldsymbol{H}_{b,t} + \boldsymbol{b}_1\big)$$

$$\alpha_{b,t} = \frac{\exp\big(e_{b,t}\big)}{\sum_{j=1}^{T} \exp\big(e_{b,j}\big)}$$

5.2 - Ricerca e selezione delle architetture candidate

La ricerca delle configurazioni ottimali è stata condotta valutando in modo sistematico le combinazioni tra le 4 **backbone** candidate (X3D-S, X3D-L, Video Swin, UniFormer-XXS), le 3 varianti di **Neck** (Identity, BiGRU, TCN) e le 2 typologie di **Head** (Mean Pooling, Attention).

5.2.1 - Feature probing: valutazione delle rappresentazioni

Una delle difficoltà nella valutazione di un'architettura multi-componente come l'architettura Backbone-Neck-Classifier, è che difficile inferire la correlazione tra un aumento o diminuzione di prestazioni del modello finale con una modifica ad un preciso componente. Un primo approccio potrebbe essere quello di fissare le configurazioni di due componenti e ottimizzare gli iperparametri per uno solo dei componenti. Il problema di questo approccio è che non si può concludere se una variazione di performance sia data maggiormente dall'iperparametro introdotto o da una mal-configurazione degli altri due componenti senza aver provato l'intero spazio di ricerca degli iperparametri. Per esempio:

> Vogliamo valutare quale tra backbone A e B sia la migliore per il task in questione. Fissiamo una neck N e un classificatore C con un set di iperparametri pre-determinato. A seguito dei test si scopre che le performance del modello A sono maggiori di quelle del modello B. Da questo dato possiamo inferire sicuramente che l'architettura A-N-C sia migliore di quella B-N-C, ma non che la backbone A sia in assoluto migliore di quella B, perché una variazione di iperparametri in N e C potrebbe favorire una rappresentazione diversa.

Per evitare questo problema abbiamo adottato un approccio basato su **probing**: le backbone verranno valutate singolarmente utilizzando semplici classificatori in downstream di diverso tipo per valutare grossolanamente la qualità della divisione delle classi già a livello di rappresentazione. Questo approccio è sensato se applicato alle backbone perché esse hanno come compito principale quello di produrre delle rappresentazioni significative e organizzate del dato di input. Tuttavia, non potendo inferire a priori la forma delle rappresentazioni abbiamo usato diversi classificatori:

- Un single layer perceptron come classificatore lineare
- Un multi layer perceptron come classificatore non lineare
- Un K-Nearest Neighbours come classificatore non lineare per testare forme clusterizzate

5.2.2 - Ricerca degli iperparametri dopo il probing

A seguito della selezione e del congelamento della backbone tramite la procedura di probing, la ricerca degli iperparametri si concentra sulla caratterizzazione e sull'ottimizzazione delle componenti successive della pipeline modulare (Neck e Head).

Lo spazio di ricerca dei parametri è definito sulle seguenti componenti.

Modulo	Opzione / Parametro	Spazio di Ricerca
Neck Architecture	BiGRU	Hidden dimension, Num layers, Dropout
	TCNN	Num channels, Kernel size, Dropout
Head Architecture	MeanPoolingClassifier	Hidden dimension, Dropout
	AttentionClassifier	Dropout
Loss Function	CrossEntropyLoss	— (*Nessun iperparametro*)
	FocalLoss	Gamma (γ)
Iperparametri Generali	Ottimizzazione	Learning rate, Weight decay

Per la ricerca dei parametri strutturali e iperparametri è stata scelta la *grid search* per la sua semplicità di esecuzione e controllo su quali parametri si volessero ottimizzare. Tuttavia lo spazio di ricerca, seppur discreto, risulta troppo grande per le risorse computazionali disponibili. Piuttosto che eseguire una ricerca esaustiva su questo spazio di ricerca, abbiamo optato per fare una ricerca stratificata dei parametri, ricercando prima quelli che a nostro parere fossero più impattanti sulle performance, fissando quelli migliori e poi procedendo per l'ottimizzazione dei parametri rimanenti. La ricerca viene quindi definita come segue:

- ***Fase 1 – Ricerca Strutturale***: Dropout e learning rate vengono fissati su un parametro predeterminato, così come la componente di classificazione resa lineare in quanto funzionante in fase di probing. In questa fase si testa quale neck è la migliore e con quali parametri.
- ***Fase 2 – Rifinitura e Regolarizzazione***: Una volta identificata la topologia ottimale del Neck dalla Fase 1, si procede all'ottimizzazione della regolarizzazione e dell'addestramento eseguendo delle run dedicate. In questa fase si testano dropout, weight decay, learning rate, un classificatore alternativo e una loss function alternativa.

I dettagli dei parametri su cui si fa ricerca sono nella seguente tabella:

Componente	Modulo / Opzione	Fase 1: Neck Search	Fase 2: Head, Loss & Tuning
Neck	BiGRU	• `hidden_dim` : [128, 256] • `num_layers` : [1, 2] • `dropout` : [0.2]	**Fissato** alla migliore configurazione emersa dalla Fase 1
	TCNNeck	• `num_channels` : [[128, 256], [256, 512], [128, 256, 512]] • `kernel_size` : [3, 5] • `dropout` : [0.2]	
Head	MeanPooling	**Fissato** (default, no extra params)	• `hidden_dims` : [[], [64], [64, 64]] • `dropout` : [0.0, 0.2]
	AttentionPooling	— (*Non valutato in Fase 1*)	• `dropout` : [0.0, 0.2]
Loss Function	CrossEntropy	**Fissata** (default)	In ricerca (nessun iperparametro extra)
	FocalLoss	— (*Non valutata in Fase 1*)	• `gamma` : [1.0, 2.0]
Ottimizzatore	AdamW / Generale	• `lr` : [5e-4] • `weight_decay` : [1e-4]	• `lr` : [5e-4] • `weight_decay` : [1e-4, 5e-5]

Table 1: Spazio di ricerca degli iperparametri suddiviso nelle due fasi di Grid Search. Le liste con valori singoli rappresentano parametri fissati.

5.2.3 - Esperimento con tracker

L'esperimento con tracking aggiunge alle feature visive un vettore temporale di 45 feature derivate dalle detection di palla e ferro; non si tratta quindi della sola concatenazione di coordinate. Il metodo di estrazione e allineamento è descritto in Section 6. Le configurazioni selezionate per i classificatori sono state ri-addestrate con le feature fuse e valutate con lo stesso protocollo. I risultati sono riportati nel capitolo successivo (Section 6).

5.2.4 - Metriche e valutazione

I modelli sono stati valutati sulla base della **macro-F1**, in quanto è una media delle singole **F1-score**. Date n classi:

$$\text{Macro-F1} = \frac{F1_1 + F1_2 + ... + F1_n}{n}, \quad F1_i = 2 \times \frac{\text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}$$

Per ogni test è anche stata riportata l'**accuracy** come metro di paragone rispetto la prima metrica: generalmente, per dataset sbilanciati, alta accuracy e bassa macro-F1 è un forte indicatore che il modello sta imparando a predire solo le classi maggioritarie. Il nostro obiettivo è quello di massimizzare la macro-F1, usando l'accuracy come primo indicatore diagnostico.

6 - Tracker palla-ferro e fusione con Video Swin

6.1 - Ruolo del tracking nel sistema

Video Swin fornisce una rappresentazione appresa del contenuto del video [13]. Il tracking le affianca informazioni esplicite su palla e ferro: presenza, posizione, movimento e relazione geometrica fra i due oggetti.

> **In sintesi**
> Il tracker **non** predice la classe dell'azione. Produce un vettore di 45 feature per ogni passo temporale, che viene concatenato all'embedding di Video Swin e passato al classificatore temporale descritto nel capitolo Section 5.

La pipeline si articola in tre passaggi. Tenerli separati permette di riutilizzare la cache delle detection e di controllare l'allineamento fra i due flussi di feature.

Passaggio	Operazione	Risultato
1. Detection	YOLO elabora i frame e conserva i candidati per palla e ferro.	Cache per clip con box, confidenze, timestamp e metadati del detector.
2. Tracking e aggregazione	Selezione temporale dei candidati e calcolo delle feature dense; aggregazione sulle finestre associate ai passi di Video Swin.	Sidecar per split, con 45 feature per passo e ID delle clip.
3. Fusione	Verifica di split, ID e lunghezza temporale; normalizzazione delle feature continue.	Embedding visivo concatenato alle feature di tracking selezionate.

6.2 - Detector: dati, annotazione e fine-tuning

Il detector è YOLO11m con fine-tuning sulle classi `ball` (ID 0) e `rim` (ID 1). Poiché i modelli pre-addestrati non conoscono il ferro e rilevano la palla da basket in modo poco affidabile a scala ridotta, abbiamo costruito un dataset di annotazione dedicato, estratto dalle stesse partite del dataset di classificazione.

6.2.1 - Selezione dei frame

I frame da annotare sono stati estratti con uno script di campionamento mirato, pensato per limitare i bias del dataset. In particolare:

1. **Nessuna contaminazione fra split.** I frame di train provengono solo dai video dello split di train, quelli di validation del detector solo dal video dello split di validation; lo split di test non viene mai usato.
2. **Quota uguale per ogni video sorgente**, in modo che il video più lungo non domini il campione.
3. **Quote per tipo di azione** all'interno di ciascun video: 40% tiri, 30% passaggi, 15% idle e 15% non-gioco. Le clip di ogni gruppo sono distribuite lungo l'istante di inizio, per coprire inquadrature diverse.
4. **Un frame per clip**, in posizione casuale. Per i tiri, nel 40% dei casi la posizione è scelta nella parte finale della clip (dal 60% al 100% della durata), dove la palla è vicina al ferro; negli altri casi, e per le altre azioni, l'intervallo va dal 5% al 95%.
5. **Ordine bilanciato.** I file sono numerati mescolando proporzionalmente video e gruppi, quindi ogni prefisso della lista (ad esempio i primi 150 frame) è già un campione bilanciato e si può interrompere l'annotazione quando serve.

I frame in cui la palla non è presente (tipicamente idle e non-gioco) non vengono scartati: restano nel dataset come esempi negativi, senza box. Lo script permette inoltre di eseguire round successivi escludendo le clip già annotate e di produrre un report di controllo del bias sulle etichette (frame per video e gruppo, presenza di palla e ferro, distribuzione della larghezza della palla in pixel).

6.2.2 - Annotazione

L'annotazione è stata eseguita a mano su CVAT: in ogni frame sono stati disegnati i box di `ball` e `rim` quando visibili, poi esportati in formato YOLO. Il dataset finale contiene 400 immagini di train e 120 di validation; fra queste, 12 immagini di train e 10 di validation non contengono alcun oggetto e fungono da sfondo.

6.2.3 - Scelta del detector: YOLO11m e YOLO26m

Prima di fissare il detector abbiamo confrontato YOLO11m e YOLO26m, due modelli di taglia `m` e circa 20 milioni di parametri, con lo stesso protocollo. Una partita (`psa_corato`) è stata esclusa dall'addestramento (300 frame di train invece di 400, di cui 10 senza oggetti). Ogni modello è stato poi valutato su due insiemi: il validation set standard (120 frame) e i 100 frame annotati della partita esclusa, che misurano la capacità di generalizzare a un'inquadratura mai vista. Gli iperparametri sono identici per i due modelli (`imgsz` 1280, batch 4, seed 42, al massimo 150 epoche con `patience` 30); le run si sono fermate per early stopping, alla migliore epoca 52 per YOLO11m e 41 per YOLO26m.

Il primo confronto usa le metriche standard di detection calcolate da Ultralytics. Un box predetto è considerato corretto se si sovrappone abbastanza al box annotato, misurato con l'IoU (*Intersection over Union*, rapporto fra area di intersezione e area di unione dei due box). Le colonne della tabella indicano:

- **Recall palla**: frazione delle palle annotate che il modello rileva, alla soglia di confidenza che massimizza l'F1.
- **mAP50 palla**: precisione media (*average precision*) sulla classe `ball`, contando una detection come corretta se l'IoU è almeno 0,5. Un valore alto significa che la palla viene trovata con pochi errori, senza richiedere un box molto accurato.
- **mAP50-95 palla**: stessa misura, ma mediata su soglie di IoU da 0,5 a 0,95 con passo 0,05. È più severa, perché premia i box che coincidono bene con quelli annotati.
- **mAP50-95 tutte**: come la precedente, mediata sulle due classi `ball` e `rim`.

Valutazione	Modello	Recall palla	mAP50 palla	mAP50-95 palla	mAP50-95 tutte
Validation	YOLO11m	0,927	0,949	0,651	0,641
Validation	YOLO26m	0,802	0,907	0,652	0,636
Partita esclusa	YOLO11m	0,669	0,747	0,406	0,462
Partita esclusa	YOLO26m	0,667	0,746	0,449	0,493

Il secondo confronto è più vicino all'uso reale nel tracker, che per ogni frame sceglie la detection più confidente. Qui un candidato è corretto se il suo centro cade entro metà della dimensione maggiore del box annotato; i candidati considerati sono quelli con confidenza almeno 0,05, cioè la stessa soglia minima usata per la cache. Le colonne indicano:

- **Recall top-1 palla**: frazione delle palle annotate per cui il candidato con la confidenza più alta è corretto. È ciò che il tracker usa quando non ha ancora una traccia.
- **Recall top-5 palla**: frazione delle palle annotate per cui almeno uno dei cinque candidati più confidenti è corretto. Il confronto con la colonna precedente mostra quante palle il detector trova ma non mette al primo posto.
- **Falsi positivi palla**: numero di frame senza palla annotata in cui il detector produce comunque un candidato palla con confidenza almeno 0,25, sul totale dei frame senza palla (24 nel validation, 16 nella partita esclusa). Più è basso, meglio è: indica quanto spesso il detector "vede" una palla dove non c'è.
- **Recall top-1 ferro**: come il recall top-1 della palla, ma per il ferro: frazione dei frame con ferro annotato (104 nel validation, 92 nella partita esclusa) in cui il candidato ferro con la confidenza più alta è corretto.

Valutazione	Modello	Recall top-1 palla	Recall top-5 palla	Falsi positivi palla	Recall top-1 ferro
Validation	YOLO11m	0,96	0,96	3/24	1,00
Validation	YOLO26m	0,91	0,91	2/24	0,99
Partita esclusa	YOLO11m	0,81	0,87	6/16	1,00
Partita esclusa	YOLO26m	0,71	0,73	2/16	1,00

Perché YOLO11m
Sul validation YOLO11m recupera più palle (recall 0,927 contro 0,802) e sulla partita esclusa mette il candidato corretto al primo posto più spesso (recall top-1 0,81 contro 0,71). Poiché il tracker parte dal candidato più confidente, questa è la proprietà che conta di più. YOLO26m ha box più precisi sulla partita esclusa (mAP50-95 pari a 0,493 contro 0,462) e meno falsi positivi sulla palla, ma ne perde di più. La scelta è quindi YOLO11m.

Entrambi i modelli peggiorano molto sulla partita esclusa: con YOLO11m il mAP50 della palla scende da 0,949 a 0,747, mentre il ferro resta quasi perfetto (0,984). La causa plausibile è la scala della palla: nella partita esclusa 67 palle su 84 misurano fra 15 e 25 pixel e 9 meno di 15, mentre nel validation 71 palle su 96 superano i 25 pixel e nessuna è sotto i 15. Il confronto va letto con cautela: ogni modello è stato addestrato una sola volta, i set di valutazione sono piccoli e la partita esclusa è una sola, quindi le differenze non sono state sottoposte a test statistici.

6.2.4 - Addestramento finale

Il detector usato negli esperimenti è YOLO11m addestrato su tutti i 400 frame di train (12 senza oggetti) e validato sui 120 di validation, a partire dal checkpoint generico `yolo11m.pt`. Gli iperparametri sono quelli in tabella; ottimizzatore e learning rate sono scelti automaticamente da Ultralytics (`optimizer = auto`) e l'augmentation è quella predefinita (mosaic, flip orizzontale con probabilità 0,5, variazioni di colore, scala fino al 50%, traslazione 10%, random erasing e RandAugment).

Parametro	Valore	Note
Modello di partenza	`yolo11m.pt`	Pesi pre-addestrati forniti da Ultralytics.
`imgsz`	1280	Risoluzione d'ingresso, alta perché la palla occupa pochi pixel.
Epoche massime	150	Con arresto anticipato.
`patience`	30	Epoche senza miglioramento sul validation prima dell'arresto.
Batch size	4	Circa 8 GB di memoria GPU.
Seed	42	Fissato, con esecuzione deterministica.

L'addestramento si è fermato per early stopping dopo 127 epoche; il miglior modello, quello usato come checkpoint `ball_rim_y11m.pt`, è dell'epoca 97. Sul validation set raggiunge precisione 0,989, recall 0,936, mAP50 pari a 0,977 e mAP50-95 pari a 0,646. Poiché il validation set serve anche a scegliere l'epoca migliore, queste metriche sono leggermente ottimistiche. Inoltre il modello finale ha visto anche i frame della partita `psa_corato`, quindi i valori della partita esclusa della sezione precedente non si applicano a esso.

La cache usata negli esperimenti è identificata da `detector_id = f7b8e8ba00da0348` e contiene 4.242 clip di train, 540 di validation e 939 di test. L'identificativo dipende dall'hash dei pesi e dai parametri d'inferenza: cambiando il detector si ottiene una cache distinta.

6.3 - Detection e selezione temporale

6.3.1 - Inferenza

Il detector è applicato ai frame decodificati di ogni clip con i parametri specificati in tabella. Per ogni frame la cache conserva fino a cinque candidati palla e due candidati ferro, ordinati per confidenza, con box in coordinate normalizzate rispetto a larghezza e altezza.

Parametro	Valore	Significato
`imgsz`	1280	Risoluzione d'ingresso del detector.
Soglia minima	0,05	Confidenza minima per salvare un candidato in cache.
Candidati per frame	5 palla, 2 ferro	Massimo numero di box conservati, per confidenza decrescente.
Batch size	2	Frame elaborati per chiamata di inferenza.

6.3.2 - Selezione dei candidati

Sulle detection in cache si applica poi una soglia di confidenza di `0.3` e la selezione procede così:

1. **Ferro**: si sceglie il candidato più confidente.
2. **Palla, primo frame o dopo un'interruzione**: si parte dal candidato più confidente.
3. **Palla, osservazioni successive**: fra i candidati compatibili con il limite di velocità si sceglie il più confidente.
4. **Gap**: se il tempo dall'ultima osservazione accettata supera `max_gap_s = 0.2`, la continuità viene azzerata e si riparte dal punto 2.

Il limite di velocità è `max_speed = 1.95` unità normalizzate al secondo, con correzione dello spostamento orizzontale per l'aspect ratio del frame.

> Il gate di velocità è una regola di plausibilità cinematica, non un modello fisico. Il tracker non interpola le detection mancanti e interrompe la continuità oltre il gap configurato. I parametri sono quelli usati negli esperimenti e non sono presentati come valori ottimali.

6.4 - Allineamento temporale

Le feature visive di riferimento usano Video Swin con sampler `edge_dense` a 32 frame, preprocessing RGB e input `stretch` a 224×224. Con questa configurazione il backbone produce 16 passi temporali.

Il tracker usa `frames_per_step = 2`: raggruppa a coppie i frame campionati e aggrega le feature dense nella finestra associata a ciascun passo di Swin. L'ultima finestra include anche i frame densi successivi all'ultimo indice campionato. Per le clip più corte, il numero di passi dipende dagli indici restituiti dal sampler.

Il merge abbina le clip per ID e verifica split e dimensione temporale; in caso di disallineamento l'elaborazione si interrompe. Prima della fusione, inoltre, il codice controlla che la lunghezza del sidecar coincida con quella del riferimento.

6.5 - Feature di tracking

Il vettore completo contiene 45 valori per passo temporale, in quattro famiglie. Le prime tre descrivono ciò che accade a livello di singolo frame, la quarta riassume la finestra associata a un passo di Video Swin. Qui interessa la logica con cui sono state costruite.

Famiglia	N.	Informazione rappresentata
Detection	12	Presenza, confidenza e geometria dei box di palla e ferro.
Cinematica della palla	8	Velocità e accelerazione, con maschere che indicano quando sono calcolabili.
Relazione palla-ferro	11	Distanza e posizione relativa, avvicinamento e indizi di attraversamento.
Aggregati di finestra	14	Visibilità e posizione nella finestra, spostamento, velocità e minima distanza palla-ferro.
Totale	**45**	**Feature esplicite per ciascun passo temporale.**

6.5.1 - Detection

Per ciascuno dei due oggetti il vettore contiene un indicatore di visibilità, la confidenza del detector e la geometria del box (centro, larghezza e altezza), tutte in coordinate normalizzate rispetto alle dimensioni del frame. Questa famiglia è la base delle altre: la cinematica, la relazione con il ferro e gli aggregati sono calcolati a partire da questi box. La confidenza viene mantenuta come feature perché permette al classificatore di pesare le rilevazioni incerte, invece di trattarle come osservazioni sicure.

6.5.2 - Cinematica della palla

Velocità e accelerazione sono stimate fra osservazioni consecutive accettate della palla, usando i timestamp dei frame, e sono espresse in unità normalizzate al secondo (e al secondo quadrato). Il vettore riporta sia le componenti orizzontale e verticale, che distinguono la fase di salita da quella di discesa lungo la parabola di un tiro, sia il modulo, che misura l'intensità del movimento indipendentemente dalla direzione. Ogni gruppo ha una maschera di validità, perché la velocità richiede due osservazioni consecutive e l'accelerazione ne richiede di più: dove non sono disponibili, il valore resta zero e la maschera vale 0.

6.5.3 - Relazione palla-ferro

Questa famiglia posiziona la palla rispetto al ferro anziché rispetto al frame. Gli scostamenti orizzontale e verticale e la distanza euclidea sono espressi in unità del box del ferro (larghezze e altezze), così che i valori restino confrontabili fra inquadrature con scala diversa. Accanto alle misure continue ci sono indicatori binari semplici: palla sopra o sotto il centro del ferro, e centro della palla dentro una regione rettangolare centrata sul ferro e ampliata dal parametro `rim_expansion`. Completano la famiglia la velocità di avvicinamento, positiva quando la palla si avvicina al ferro, e l'indizio di attraversamento `ball_crossed_rim_down`, che vale 1 quando la palla passa da sopra a sotto il centro del ferro entro una tolleranza orizzontale configurata. Come per la cinematica, ogni calcolo che richiede più osservazioni ha la propria maschera di validità.

6.5.4 - Aggregati di finestra

Il tracker lavora sui frame densi, mentre Video Swin produce un solo embedding per passo temporale. Gli aggregati servono a riassumere, per ciascun passo, ciò che è accaduto nella finestra di frame ad esso associata. Descrivono quanto a lungo la palla è stata visibile, dove è stata vista per ultima, quanto si è spostata nel tempo e con quale velocità massima, quanto si è avvicinata al ferro nel punto più vicino e se in qualche frame è stato rilevato un attraversamento. Due feature indicano inoltre da quanto tempo la palla non viene rilevata e se è già comparsa prima: permettono di distinguere una palla temporaneamente occlusa da una che non è ancora entrata in scena. Anche qui, ogni aggregato è accompagnato da una maschera che segnala quando nella finestra non c'erano dati sufficienti per calcolarlo.

6.5.5 - Normalizzazione e valori mancanti

Indicatori binari e maschere restano 0/1. Le feature continue sono standardizzate con media e deviazione standard calcolate **solo sul train** e riutilizzate su validation e test, per evitare data leakage.

Quando un valore non è valido, il merge conserva zero e non applica la standardizzazione. La maschera associata distingue così il dato mancante da uno zero realmente osservato. Le assenze non vengono interpolate né propagate.

6.6 - Configurazioni di fusione

Per ogni passo temporale t, il merge concatena l'embedding visivo $\boldsymbol{h}_t$ con il vettore di tracking $\boldsymbol{z}_t$:

$$\boldsymbol{x}_t = (\boldsymbol{h}_t, \boldsymbol{z}_t) \in \boldsymbol{R}^{768+k}$$

Il numero di passi temporali resta invariato; cambia solo la dimensione del vettore. Il valore 768 è la larghezza dell'embedding di Video Swin, non una risoluzione dei frame. Le ablation `ball_only` e `rim_only` selezionano sottoinsiemi diversi del medesimo schema.

Configurazione	k	Dimensione	Contenuto
`rim_only`	6	774	Visibilità, confidenza e geometria del box del ferro.
`ball_only`	25	793	Detection e cinematica della palla, più aggregati della palla non relativi al ferro.
`all`	45	813	Detection, cinematica, relazione palla-ferro e aggregati temporali.

Ad esempio, `all` produce a ogni passo un vettore di 813 valori: 768 visuali e 45 di tracking. Il merge registra gli ID di detector e tracker, la configurazione Video Swin, lo schema delle feature e `frames_per_step`, rendendo verificabile l'origine di ogni fusione.

6.7 - Limiti e riproducibilità

Il segnale di tracking dipende dalla qualità delle detection: occlusioni, box errati o mancate rilevazioni introducono rumore. Le maschere rendono esplicite le assenze, ma non le eliminano. Anche il dataset di annotazione è di dimensioni contenute (520 frame): sulla partita esclusa dall'addestramento la rilevazione della palla peggiora sensibilmente, in particolare quando la palla è piccola, e questo limita la robustezza del detector su inquadrature non rappresentate.

> **Attenzione all'interpretazione**
> L'indizio `ball_crossed_rim_down` descrive un attraversamento **geometrico** della regione del ferro: da solo non dimostra che il tiro sia stato realizzato.

Il confronto fra YOLO11m e YOLO26m è stato condotto solo a livello di detector: non sono state rigenerate cache, feature e merge con YOLO26m, quindi non sappiamo se la differenza osservata si rifletterebbe sulle prestazioni di classificazione. Le conclusioni valgono per la pipeline YOLO11m effettivamente usata.

7 - Risultati

In questo capitolo vengono presentati e analizzati i risultati empirici ottenuti dalla valutazione del framework modulare BNC e della strategia di classificazione gerarchica. L'analisi è strutturata in conformità con il protocollo sperimentale definito nel Capitolo 4, distinguendo nettamente la fase di probing da quella di ricerca degli iperparametri e selezione del modello condotta sul Validation set dalle valutazioni finali ed esenti da contaminazione effettuate sul Test set.

I migliori modelli risultanti dalla grid search sono riportati nella tabella Table 10 assieme ai risultati di validazione.

7.1 - Risultati del probing

Date le backbone selezionate in Section 5, sono state provate le seguenti configurazioni di pre-processing sulle clip:

ID	Sampling	Number of frames	Spatial Mode
A	Edge Dense	16	letterbox
B	Edge Dense	16	stretch
C	Edge Dense	32	stretch
D	Uniform	16	letterbox
E	Uniform	16	stretch
F	All	-	stretch

Table 2: Parametri di pre-processing video per probing

Mentre i modelli di probing scelti sono stati:

Model	Parameters
Linear	-
Multi-Layer Perceptron	2 layers, 256 hidden dim
Multi-Layer Perceptron	3 layers, 256 hidden dim
K-Nearest Neighbours	K=3

Table 3: Modelli usati per il probing

I risultati sono stati ottenuti mediando le performance ottenute con 5 run con seed progressivi a partire da 42 per ogni configurazione della Table 2, per ogni modello della Table 3 addestrati sul train set e validati sul validation set e per il dataset etichettato come d'origine e stratificato in fasi. Di seguito verranno riportati gli esperimenti di probing identificando le feature di addestramento in base alle modalità di pre-processing come riportato nella colonna **ID** di Table 2.

7.1.1 - ConvNext2D

Phase	Prober	A	B	C	D	E	F
flat	MLP_L1_H256	0.2805	0.2786	0.3312	0.2561	0.2801	0.2608
	MLP_L2_H256	0.3108	0.3331	**0.3884**	0.2990	0.3402	0.3227
	MLP_L3_H256	0.2860	0.3283	0.3447	0.2761	0.3220	0.3666
	kNN_Probe_k3_cosine	0.2197	0.2131	0.2642	0.1903	0.1830	0.1912
p1	MLP_L1_H256	0.7042	0.6969	0.6857	0.6674	0.6432	0.6437
	MLP_L2_H256	0.6966	0.7005	**0.7499**	0.6707	0.6927	0.6969
	MLP_L3_H256	0.6982	0.6952	0.7436	0.6513	0.6960	0.6807
	kNN_Probe_k3_cosine	0.6171	0.6123	0.6059	0.5442	0.5880	0.5875
p2	MLP_L1_H256	0.6146	0.6609	0.6404	0.6182	0.6836	0.6646
	MLP_L2_H256	0.6429	0.6273	0.6261	0.6734	0.6644	0.6876
	MLP_L3_H256	0.6249	0.6267	0.6521	**0.6919**	0.5975	0.6365
	kNN_Probe_k3_cosine	0.4032	0.5251	0.5336	0.4291	0.6127	0.5524
p3	MLP_L1_H256	0.6292	0.5476	0.5691	**0.6565**	0.5972	0.5738
	MLP_L2_H256	0.6329	0.5228	0.5647	0.6289	0.5912	0.5949
	MLP_L3_H256	0.6109	0.5443	0.5352	0.6463	0.5762	0.5464
	kNN_Probe_k3_cosine	0.6091	0.5788	0.5774	0.5834	0.5774	0.5619

Table 4: Confronto Macro F1-Score per Dataset e Prober, in bold le migliori prestazioni per fase

7.1.2 - UniFormer-XXS

Phase	Prober	A	B	C	D	E	F
flat	MLP_L1_H256	0.4445	0.4405	0.4501	0.3692	0.3730	0.4211
	MLP_L2_H256	0.4199	0.4222	**0.4956**	0.3502	0.4276	0.4078
	MLP_L3_H256	0.4308	0.4508	0.4551	0.3834	0.4195	0.4110
	kNN_Probe_k3_cosine	0.3412	0.3494	0.4129	0.3401	0.3479	0.2828
p1	MLP_L1_H256	0.8300	0.8437	0.8634	0.7991	0.8539	0.8563
	MLP_L2_H256	0.8306	0.8308	0.8814	0.8142	0.8587	0.8589
	MLP_L3_H256	0.8439	0.8559	**0.8845**	0.8311	0.8460	0.8574
	kNN_Probe_k3_cosine	0.7826	0.7748	0.8250	0.7762	0.7428	0.7616
p2	MLP_L1_H256	**0.7393**	0.7037	0.6801	0.6224	0.6646	0.6588
	MLP_L2_H256	0.6787	0.7036	0.6653	0.6373	0.6462	0.6845
	MLP_L3_H256	0.6434	0.6836	0.6449	0.6037	0.6522	0.6766
	kNN_Probe_k3_cosine	0.4812	0.4762	0.5174	0.4573	0.5789	0.5649
p3	MLP_L1_H256	0.6416	0.5842	0.6799	0.6213	0.5615	0.6450
	MLP_L2_H256	0.6656	0.5708	0.7539	0.6454	0.6246	0.6670
	MLP_L3_H256	0.6914	0.5937	**0.7549**	0.6330	0.5670	0.6206
	kNN_Probe_k3_cosine	0.5752	0.5166	0.5538	0.5788	0.5906	0.5534

Table 5: CConfronto Macro F1-Score per Dataset e Prober, in bold le migliori prestazioni per fase

7.1.3 - VideoSwin

Phase	Prober	A	B	C	D	E	F
flat	MLP_L1_H256	0.4525	0.4776	0.4664	0.4348	0.4886	0.4730
	MLP_L2_H256	0.4565	0.4264	0.4704	0.5216	0.5047	0.4504
	MLP_L3_H256	0.4518	0.4325	0.5009	**0.5354**	0.4747	0.4531
	kNN_Probe_k3_cosine	0.3374	0.2778	0.3260	0.3352	0.2999	0.2657
p1	MLP_L1_H256	0.8638	0.8607	0.8806	0.8281	0.8461	0.8586
	MLP_L2_H256	0.8755	0.8708	0.8821	0.8694	0.8805	0.8672
	MLP_L3_H256	0.8770	0.8777	**0.8848**	0.8676	0.8740	0.8740
	kNN_Probe_k3_cosine	0.7533	0.7584	0.7458	0.7861	0.7610	0.7508
p2	MLP_L1_H256	0.6279	0.7691	0.8089	0.7430	0.7898	**0.8543**
	MLP_L2_H256	0.6430	0.7557	0.8076	0.7391	0.8086	0.8086
	MLP_L3_H256	0.7043	0.7342	0.8331	0.7385	0.7851	0.7842
	kNN_Probe_k3_cosine	0.5714	0.5246	0.5075	0.5698	0.5236	0.5397
p3	MLP_L1_H256	0.5655	0.6514	0.5587	0.6391	0.6046	0.5888
	MLP_L2_H256	0.5864	0.5668	0.5444	**0.6795**	0.6591	0.5953
	MLP_L3_H256	0.5821	0.5671	0.5423	0.6471	0.6592	0.5931
	kNN_Probe_k3_cosine	0.5290	0.5290	0.5190	0.5192	0.5877	0.5046

Table 6: CConfronto Macro F1-Score per Dataset e Prober, in bold le migliori prestazioni per fase

7.1.4 - X3D-S

Phase	Prober	A	B	C	D	E	F
flat	MLP_L1_H256	0.3241	0.4265	0.4135	0.3482	0.4315	0.4284
	MLP_L2_H256	0.3875	0.4413	0.4297	0.3429	0.4551	0.4610
	MLP_L3_H256	0.3472	0.4472	0.4323	0.4086	0.4262	**0.4611**
	kNN_Probe_k3_cosine	0.2964	0.3541	0.3728	0.2539	0.3821	0.3142
p1	MLP_L1_H256	0.7325	0.8088	0.8491	0.7191	0.8099	0.8436
	MLP_L2_H256	0.7792	0.8237	0.8413	0.7791	0.8378	0.8495
	MLP_L3_H256	0.7642	0.8260	0.8545	0.7904	0.8299	**0.8559**
	kNN_Probe_k3_cosine	0.6901	0.7903	0.8126	0.7087	0.7824	0.7805
p2	MLP_L1_H256	0.6660	0.6882	0.6816	0.7280	0.6677	0.7207
	MLP_L2_H256	0.6646	0.6989	0.7374	0.7003	**0.7610**	0.7384
	MLP_L3_H256	0.6100	0.7047	0.7438	0.6609	0.7606	0.7261
	kNN_Probe_k3_cosine	0.4619	0.5178	0.4872	0.4510	0.6089	0.5224
p3	MLP_L1_H256	0.6035	0.5305	0.5726	0.6880	0.4930	0.4718
	MLP_L2_H256	0.6101	0.5863	**0.7004**	0.6925	0.6219	0.5338
	MLP_L3_H256	0.5815	0.5490	0.6499	0.6525	0.6148	0.5718
	kNN_Probe_k3_cosine	0.5752	0.5656	0.5655	0.4478	0.4953	0.5538

Table 7: Confronto Macro F1-Score per Dataset e Prober, in bold le migliori prestazioni per fase

7.1.5 - X3D-L

Phase	Prober	A	B	C	D	E	F
flat	MLP_L1_H256	0.4206	0.3817	0.4458	0.3627	0.4672	0.4268
	MLP_L2_H256	0.4275	0.4075	0.4326	0.3405	**0.5161**	0.4523
	MLP_L3_H256	0.4182	0.4309	0.4545	0.3285	0.4609	0.4417
	kNN_Probe_k3_cosine	0.3135	0.3071	0.4246	0.3474	0.3371	0.3122
p1	MLP_L1_H256	0.7741	0.8149	0.8403	0.7592	0.7996	0.7792
	MLP_L2_H256	0.8033	0.8488	**0.8649**	0.7693	0.8185	0.7184
	MLP_L3_H256	0.7959	0.8393	0.8605	0.7857	0.8135	0.7038
	kNN_Probe_k3_cosine	0.7441	0.7763	0.8136	0.6917	0.7334	0.5053
p2	MLP_L1_H256	0.6410	0.7106	0.6835	0.6186	0.7875	0.8102
	MLP_L2_H256	0.6399	0.7735	0.7065	0.6170	0.8011	**0.8387**
	MLP_L3_H256	0.5953	0.7290	0.6924	0.5915	0.7887	0.7909
	kNN_Probe_k3_cosine	0.5157	0.5303	0.6184	0.5714	0.5966	0.4932
p3	MLP_L1_H256	0.6832	0.6172	0.6744	0.5787	0.5571	0.6116
	MLP_L2_H256	0.6586	0.6277	**0.6901**	0.6101	0.6006	0.6365
	MLP_L3_H256	0.6665	0.6321	0.6535	0.5745	0.6078	0.6283
	kNN_Probe_k3_cosine	0.5501	0.5026	0.6154	0.6221	0.4923	0.6168

Table 8: Confronto Macro F1-Score per Dataset e Prober, in bold le migliori prestazioni per fase

7.1.6 - Confronto

Il dataset F, quindi con sampling totale, è stato incluso come *oracolo*, ma escluso dal confronto finale in quanto avrebbe portato i modelli ad affidarsi al numero di frame della clip per fare classificazione, all'effettivo un data leakage. Condensando le altre rappresentazioni otteniamo la seguente tabella:

Fase	Backbone	Migliori prestazioni	Dataset migliore
P1	ConvNext2D	75%	C
	UniFormer-XXS	**88%**	C
	VideoSwin	**88%**	C
	X3D-S	85%	C-F
	X3D-L	86%	C
P2	ConvNext2D	69%	D
	UniFormer-XXS	73%	A
	VideoSwin	**85%-83%**	F-C
	X3D-S	76%	E
	X3D-L	**83%**	F
P3	ConvNext2D	65%	D
	UniFormer-XXS	**75%**	C
	VideoSwin	67%	D
	X3D-S	70%	C
	X3D-L	69%	C

Table 9: Riepilogo delle migliori prestazioni delle backbone per fase e dataset.

Visti i risultati di Table 9 è stata selezionata **VideoSwin**, che ottiene i migliori risultati sulle prime due fasi e dei buoni risultati sulla terza. La configurazione di pre-processing più efficace è risultata la C, quindi con sampling *Edge Dense* a 32 frame con spacial mode *stretch*.

7.2 - Risultati senza tracker

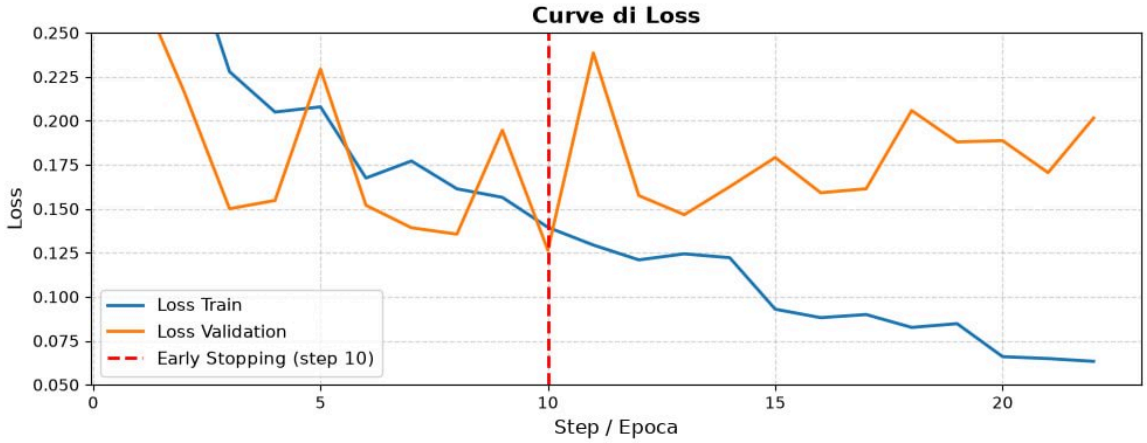
Fase	Configurazione Ottimale (Neck)	Val Loss	Val Accuracy	Val Macro F1
p_1 (Macro)	BiGRU (hidden_dim: 128, num_layers: 1, dropout: 0.2) MeanPooling (hidden_dims: [64, 64], dropout: 0) Loss: FocalLoss(gamma: 2) Learning rate: 10^{-4}, Weight Decay: 10^{-5}	0.1276 ± 0.0037	0.8926 ± 0.0059	0.8993 ± 0.0077
p_2 (Tipo)	BiGRU (hidden_dim: 256, num_layers: 1, dropout: 0.2) AttentionPooling (dropout: 0.2) Loss: FocalLoss(gamma: 2) Learning rate: $5 \cdot 10^{-4}$, Weight Decay: $5 \cdot 10^{-5}$	0.0708 ± 0.0041	0.8892 ± 0.0069	0.8844 ± 0.0057
p_3 (Esito)	BiGRU (hidden_dim: 256, num_layers: 2, dropout: 0.2) AttentionPooling (dropout: 0) Loss: FocalLoss(gamma: 2) Learning rate: $5 \cdot 10^{-4}$, Weight Decay: $5 \cdot 10^{-5}$	0.1542 ± 0.0037	0.7108 ± 0.0413	0.6826 ± 0.0413
Flat	BiGRU (hidden_dim: 256, num_layers: 2, dropout: 0.2) MeanPooling (hidden_dims: [], dropout: 0) Loss: CrossEntropy Learning rate: $5 \cdot 10^{-4}$, Weight Decay: 10^{-4}	0.8795 ± 0.0273	0.7574 ± 0.0100	0.5199 ± 0.0377

Table 10: Sintesi delle migliori configurazioni architetturali emerse dalla ricerca su griglia e relative prestazioni medie (con deviazione standard) sulla partizione di Validation.

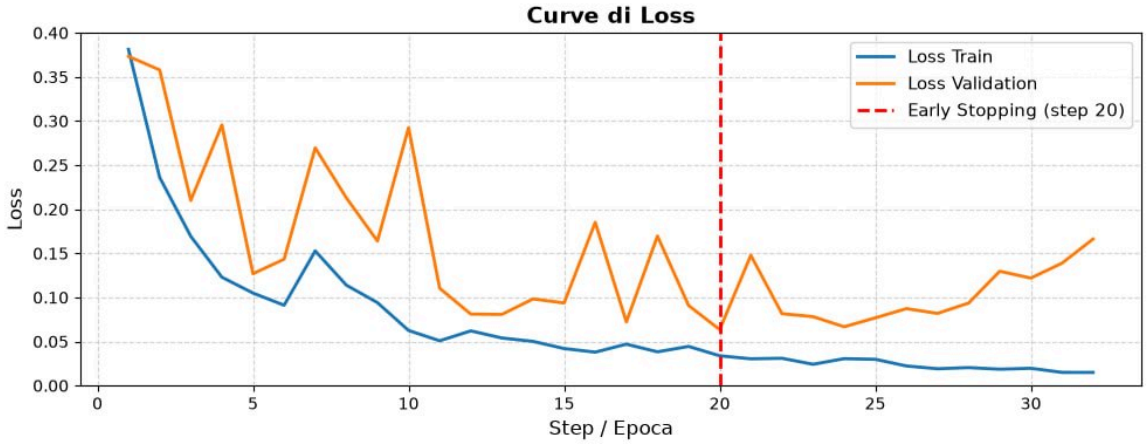
In tabella Table 11 sono stati riportati i risultati dei migliori modelli trovati con la grid search sulla fold di test riportando accuracy e macro F1. L'ultimo test espone invece il modello in downstream, quindi la somma dei tre modelli addestrati precedentemente.

Fase	Task Target	Test Loss	Test Accuracy	Test Macro F1
p_1	Riconoscimento Macro-Azione	0.4312	0.8541	0.8423
p_2	Tipologia del Tiro	0.4420	0.7312	0.7133
p_3	Esito del Tiro	0.1692	0.6250	0.5943
Flat	Label originali	1.5004	0.6624	0.4524
Downstream $p_{1\to2\to3}$	Label originali	-	0.7710	0.4477
Downstream $p_{1\to2}$	Label originali, escluso esito tiro	-	0.8094	0.6841

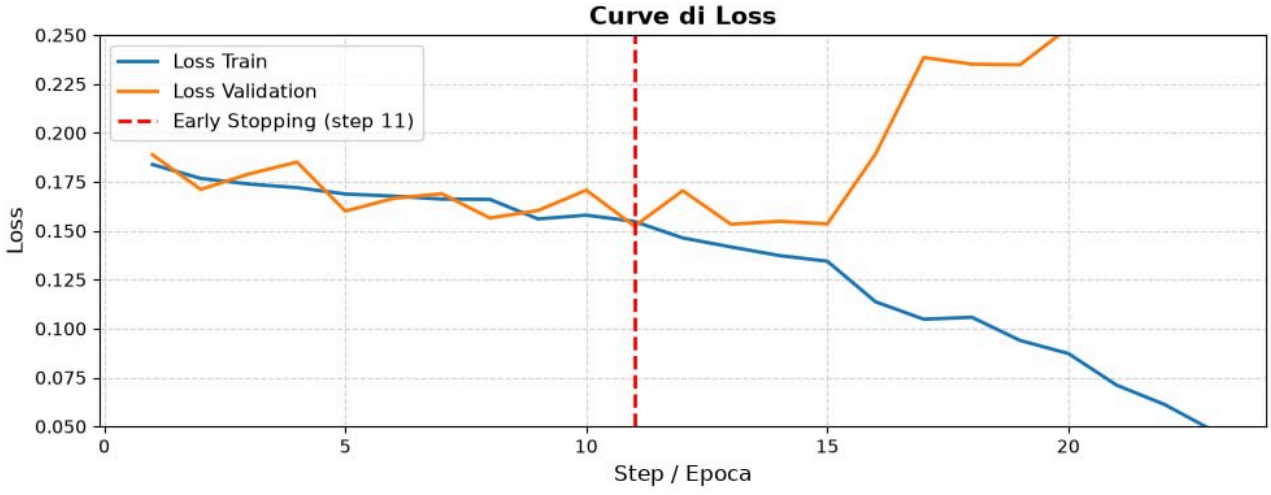
Table 11: Loss, accuracy e macro F1 sulla fold di test divise per fase. La riga *Flat* coentiene le prestazioni del miglior modello baseline, quindi senza stratificazione delle label. L'ultima riga è il modello completo lanciato in downstream, prima provando la pipeline completa e poi escludendo l'inferenza di p_3



(a) Fase 1 (p_1): Macro-Azione



(b) Fase 2 (p_2): Tipologia Tiro



(c) Fase 3 (p_3): Esito Tiro

Figure 2: Andamento delle curve di loss di addestramento e validazione lungo le tre fasi della gerarchia (p_1, p_2, p_3) per i modelli ottimali selezionati.

La Figure 2 mette a confronto le curve di loss in addestramento e validazione per le migliori configurazioni architetturali individuate tramite grid search, disaggregate per le tre fasi della cascata gerarchica (p_1, p_2, p_3). L'addestramento è stato regolato da un meccanismo di Early Stopping con una tolleranza (patience) di 12.

7.3 - Risultati con tracker

In Table 12 i risultati ri-addestrando i modelli con le migliori configurazioni date dalla grid search con aggiunta di feature di tracking.

Fase	Configurazione Ottimale	Val Loss	Val Accuracy	Val Macro F1	Delta
p_1	BiGRU (hidden_dim: 128, num_layers: 1, dropout: 0.2) MeanPooling (hidden_dims: [64, 64], dropout: 0) Loss: FocalLoss(gamma: 2) Learning rate: 10^{-4}, Weight Decay: 10^{-5}	0.0822 ± 0.0043	0.8866 ± 0.0038	0.8940 ± 0.0049	-0.0054
p_2	BiGRU (hidden_dim: 256, num_layers: 1, dropout: 0.2) AttentionPooling (dropout: 0.2) Loss: FocalLoss(gamma: 2) Learning rate: $5 \cdot 10^{-4}$, Weight Decay: $5 \cdot 10^{-5}$	0.1475 ± 0.0119	0.7846 ± 0.0407	0.7717 ± 0.0396	-0.1127
p_3	BiGRU (hidden_dim: 256, num_layers: 2, dropout: 0.2) AttentionPooling (dropout: 0) Loss: FocalLoss(gamma: 2) Learning rate: $5 \cdot 10^{-4}$, Weight Decay: $5 \cdot 10^{-5}$	0.0823 ± 0.0160	0.8523 ± 0.0233	0.8426 ± 0.0250	$+0.1600$
Flat	BiGRU (hidden_dim: 256, num_layers: 2, dropout: 0.2) MeanPooling (hidden_dims: [], dropout: 0) Loss: CrossEntropy Learning rate: $5 \cdot 10^{-4}$, Weight Decay: 10^{-4}	0.5697 ± 0.0187	0.8341 ± 0.0110	0.7569 ± 0.0372	$+0.2370$

Table 12: Sintesi delle migliori configurazioni architetturali (le stesse dell'esperimento precedente) con aggiunta di tracker. La colonna *Delta* rappresenta la variazione di performance avuta dall'aggiunta del tracker.

In Table 13 uno studio di ablazione sulle feature del tracker.

Phase	ALL Accuracy	ALL MacroF1	BALL_ONLY Accuracy	BALL_ONLY MacroF1	RIM_ONLY Accuracy	RIM_ONLY MacroF1
Phase P1	0.8866 ±0.0038	**0.8941 ±0.0049**	0.8674 ±0.0088	0.8693 ±0.0109	0.8693 ±0.0064	0.8700 ±0.0075
Phase P2	0.7846 ±0.0407	**0.7717 ±0.0396**	0.7692 ±0.0288	0.7588 ±0.0274	0.6584 ±0.0537	0.6349 ±0.0325
Phase P3	0.8523 ±0.0233	**0.8426 ±0.0250**	0.8092 ±0.0469	0.7942 ±0.0493	0.6523 ±0.0416	0.6369 ±0.0359
Phase FLAT	0.8341 ±0.0110	**0.7569 ±0.0372**	0.7952 ±0.0207	0.6416 ±0.0405	0.7396 ±0.0282	0.4538 ±0.0459

Table 13: Accuracy e macro-F1 sul validation set per feature tracking totali (ALL), solo tracking della palla (BALL_ONLY) e solo traching del canestro (RIM_ONLY). In grassetto le migliori macro-F1

E infine, i modelli singoli e downstream con aggiunta di tracking e calcolo della differenza (Delta) rispetto alle performance dei modelli su test set prima dell'aggiunta di tracking, visionabili in Table 14. Anche in questo caso c'è stata una sostanziale perdita di performance in fase p_2 ma una rilevante miglioria in fase p_3. In downstream, la fase p_1 e p_2 sono state fissate a quelle senza tracker, in quanto nel primo caso la differenza di prestazione è trascurabile e nel secondo è peggiorata, mentre per p_3 verrà utilizzato il modello addestrato con tracking.

Fase	Test Loss	Test Accuracy	Test Macro F1	Delta
p_1	0.3085	0.8530	0.8501	+0.0078
p_2	0.4277	0.6937	0.6362	−0.0771
p_3	0.1522	0.8063	0.8062	+0.2119
Downstream $p_{1\to2\to3}$	-	0.8051	0.5898	+0.1421
Downstream $p_{1\to2}$	-	0.8094	0.6841	-

Table 14: Loss, accuracy e macro F1 sulla fold di test divise per fase a seguito dell'aggiunta di tracking. Le ultime due righe sono rispettivamente il modelli completo lanciato in downstream e il modello completo escludendo l'inferenza di p_3

Per una migliore interpretazione dei risultati, di seguito vengono riportate le confusion matrices relative alle misurazioni sul test set.

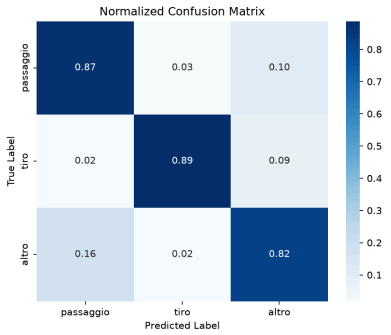


Figure 4: p_1

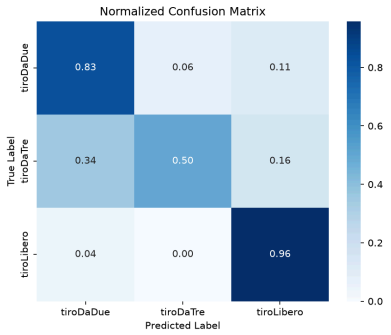


Figure 5: p_2

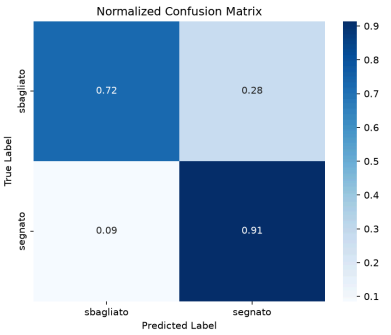


Figure 6: p_3

Figure 3: Confusion matrices dei tre migliori modelli valutati singolarmente sul test set. I primi due senza tracker, l'ultimo con tracker.

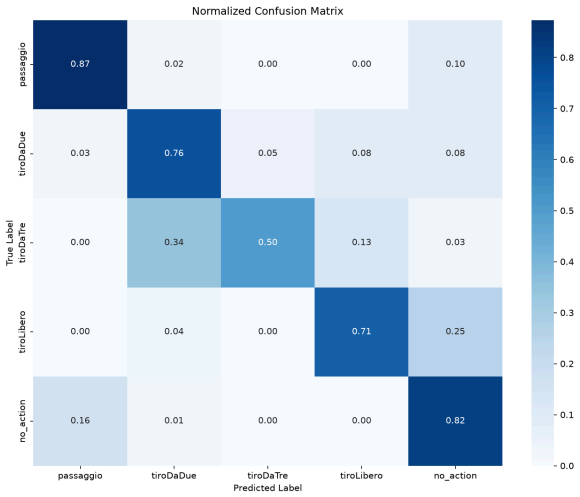


Figure 8: Confusion matrix downstream $p_1 \rightarrow p_2$

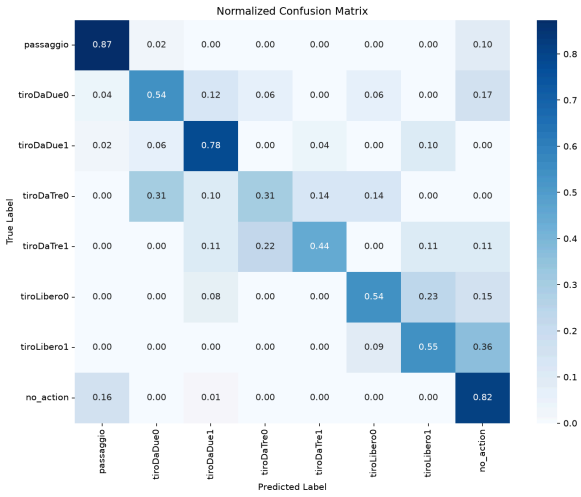


Figure 9: Confusion matrix downstream $p_1 \rightarrow p_2 \rightarrow p_3$

Figure 7: Confusion matrices modello in downstream

8 - Discussione

Dai risultati in Table 10 si evince che il task di discriminazione dell'esito dei tiri è il più difficile, raggiungendo una macro-f1 solo del 68%. In fase di test tutti i modelli hanno avuto una ricaduta nelle metriche sostanziale: P1 −5%, P2 −17%, P3 −9%. L'ensamble di modelli posti in downstream performa solo per il 44% di macro-F1. Considerando il notevole crollo di prestazioni è necessario fare alcune osservazioni per trovare la causa. Nella Section 3.5 è stata fatta una proiezione UMAP per evidenziale una potenziale problematica strutturale nei dati causata dalla forte correlazione tra clip e la partita relativa. I risultati ottenuti sono coerenti con questa interpretazione: le alte performance in validation rispecchiano la sovrapposizione nello spazio topologico tra la fold di validation set e una delle partite del training set. Al coltempo escludiamo problematiche relative ad overfitting considerando l'adoperazione di early stopping e le curve Figure 2. L'unica spiegazione plausibile è che ci sia stato un forte domain shift nel test set, ipotesi rafforzata dalla proiezione in questione, dove si vede chiaramente che un'intera regione del test set è completamente scollagata dal training set.

La nostra conclusione è che quindi con una rappresentazione puramente visuale, le backbone siano troppo sensibili a piccole variazioni di inquadratura e risoluzione. Da Table 11 possiamo anche confermare che il task p_3 è il più complesso, causando un crollo delle metriche del 21% in downstream, marcando la necessità di curare meglio le feature in input al modello per azioni dettagliate. A tal proposito è stato proposto un sistema di tracking come spiegato in Section 6. Da Table 12 vediamo una variazione trascurabile nel task p_1, una riduzione sostanziale in p_2, ma una sostanziale miglioria sul task p_3, portando una miglioria del 16% su macro-F1 in validation.

Singolarmente, le prestazioni di questi modelli sono visualizzabili come *confusion matrices* in Figure 3. Da queste si evince che il modello fatica particolarmente nella distinzione *tiroDaTre* e *tiroDaDue*. Questo potrebbe essere dato dal fatto che i giocatori tendono a raccogliersi il più vicino possibile alla linea dei tre punti per un tiro lungo e non sempre direttamente sotto il canestro per un tiro corto, mentre i tiri liberi sono quasi perfettamente classificati. La bassa capacità discriminativa tra le due classi in questione è molto probabilmente la causa più sostanziale dell'accumulo di errore nel processo in downstream.

Per finalizzare l'esperimento, questi tre modelli sono stati predisposti ad un'inferenza in downstream, i quali risultati sono esposti in Table 14. Rispetto il processo downstream completamente senza tracking c'è stato un aumento di circa il 14% sulla macro-F1 in downstream sul test set, portandola a $\approx 59\%$ e battendo la baseline (45%) di 14 punti sulla macro-F1.

9 - Conclusioni

L'approccio incrementale sostenuto durante la sperimentazione ha reso chiare quali componenti fossero più incidenti sulla classificazione, isolando singolarmente le componenti in studio, permettendo di sostenere un ragionamento ad ablazione seguendo l'ordine degli esperimenti a ritroso. Se per esempio ci si chiedesse quanto ha influito l'aggiunta della neck nell'architettura sarebbe sufficiente consultare gli esperimenti di probing, che per l'appunto mancano della neck.

Spostandoci sul metodo, la scelta di stratificare l'etichettatura del dataset si è rivelata vincente, permettendoci di esplorare combinazioni di neck e classificatore leggere. L'unico svantaggio consiste nella complicazione della pipeline e nella necessità di condurre una ricerca parametrica diversa per ogni task, potenzialmente terminando in scelte implementative differenti per ogni fase.

Nel complesso, la metodologia proposta presenta buone performance in inferenza, sovrastate tuttavia dalle osservazioni sulla limitatezza del dataset esposte in Section 3.5, da attribuire più ad una mancanza di dati che a problematiche relative all'etichettatura del dataset o di acquisizione dei video. La variabilità tra le partite è una proprietà che un modello di action recognition per lo sport dovrebbe superare, ma abbiamo constatato che è necessario una quantità di dati molto maggiore. Un'altra ipotesi avanzata durante lo svolgimento del progetto è che le backbone sarebbero state inadatte a questo tipo di task, tuttavia ulteriori ricerche hanno rivelato che altre backbone dello stato dell'arte non usano meccanismi di apprendimento troppo diversi da quelle selezionate, rendendo improbabile che la scelta di altre backbone avrebbe migliorato l'estrazione delle feature.

In futuro, si potrebbe esplorare la possibilità di addestrare una backbone apposita, con l'obiettivo esplicito di farle apprendere una rappresentazione invariante dall'angolatura della videocamera o la risoluzione. Un'altro potenziale sviluppo futuro potrebbe risiedere nello sviluppo di un modello che estragga feature spaziali esplicite riguardo la posizione dei giocatori nel campo, o proprio delle linee di campo, con l'obiettivo di migliorare la capacità discriminativa dei tiri.

Bibliography

[1] W. Kay *et al.*, "The Kinetics human action video dataset," *arXiv preprint arXiv:1705.06950*, 2017.

[2] D. Shao, Y. Zhao, B. Dai, and D. Lin, "FineGym: A Hierarchical Video Dataset for Fine-Grained Action Understanding," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 2613–2622. doi: 10.1109/CVPR42600.2020.00269.

[3] P. Gong and X. Luo, "A survey of video action recognition based on deep learning," *Knowledge-Based Systems*, vol. 320, p. 113594, 2025.

[4] F. Caba Heilbron, V. Escorcia, B. Ghanem, and J. Carlos Niebles, "ActivityNet: A Large-Scale Video Benchmark for Human Activity Understanding," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 961–970.

[5] Z. Shou, D. Wang, and S.-F. Chang, "Action Segment Candidate Generation for Action Detection in Untrimmed Videos," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1937–1946.

[6] K. Soomro, A. R. Zamir, and M. Shah, "UCF101: A Dataset of 101 Human Actions Classes From Videos in The Wild," *arXiv preprint arXiv:1212.0402*, 2012.

[7] H. Wang, A. Käser, C. Schmid, and C.-L. Liu, "Action recognition by dense trajectories," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2011, pp. 3169–3176.

[8] M. Zhai, X. Xiang, N. Lv, and X. Kong, "Optical flow and scene flow estimation: A survey," *Pattern Recognition*, vol. 114, p. 107861, 2021.

[9] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[10] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A convnet for the 2020s," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 11976–11986.

[11] J. Carreira and A. Zisserman, "Quo vadis, action recognition? A new model and the kinetics dataset," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 6299–6308.

[12] C. Feichtenhofer, "X3D: Expanding architectures for efficient video recognition," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 203–213.

[13] Z. Liu *et al.*, "Video Swin Transformer," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 3202–3211.

[14] K. Li *et al.*, "UniFormer: Unified Transformer for Efficient Spatial-Temporal Representation Learning," in *International Conference on Learning Representations (ICLR)*, 2022.

[15] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[16] K. Cho *et al.*, "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," in *Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.

[17] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," *arXiv preprint arXiv:1803.01271*, 2018.

[18] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[19] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.